

# Software Engineering Interview Playbook

Sep 29, 2026 · @Komol Chandra Devnath

## Start here

Every story in this playbook follows the same shape: a 60-second pitch, the drill questions you will get, a 4-step answer for each, then the trade-off and numbers to say out loud. Read the cheat sheet the night before; practice the stories aloud with the plan at the end.

## How each story block is laid out

1. **Pitch (STAR)** — Situation, Task, Action, Result in about 90 seconds.
2. **Core concepts** — the technical depth an interviewer will probe.
3. **Likely drill questions** — with a model answer in the 4-step formula.
4. **Say it** — trade-off lines, numbers to know, key phrases.
5. **[FILL]** — facts only you can confirm before saying them.

## 60-second cheat sheet

| Story                       | Headline number                               | One line to remember                                         |
|-----------------------------|-----------------------------------------------|--------------------------------------------------------------|
| C1 Microservices            | ~20% lower core CPU, 0 downtime, 100+ tenants | Boundaries drawn around scaling needs and data ownership     |
| C2 Query optimization       | Up to 85% faster heaviest report              | "Up to" = best case, not average                             |
| C3 Incident response        | ~65% lower MTTR                               | Alert on user symptoms, not raw CPU                          |
| C4 Frontend rewrite         | 2 months, ~30% engagement lift                | Foundation first, core journeys first                        |
| C5 Tenancy / payments / POS | ~60% less manual reconciliation               | Webhooks are the fast path; reconciliation is the safety net |
| C6 Enterprise API           | Impact → Cause → Action → Time                | Client UUIDs make retries safe                               |
| C7 Field force / GPS        | 50 m geofence, >150 km/h                      | Flag for review, don't auto-                                 |

| Story      | Headline number                 | One line to remember                                                   |
|------------|---------------------------------|------------------------------------------------------------------------|
|            | velocity flag                   | reject                                                                 |
| C8 Node.js | Microtasks before<br>macrotasks | Never block the event loop                                             |
| C9 CI/CD   | ~40% faster deploys             | Tag images by commit SHA;<br>migrations must be<br>backward-compatible |

## 1. The answer method

US interviewers want the conclusion first. Answer every technical follow-up in four steps, about 30–60 seconds total.

| Step             | Length        | What to say                       | Example                                                                                                              |
|------------------|---------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------|
| 1. Direct answer | 1 sentence    | The decision or conclusion        | "We used a shared database during the transition."                                                                   |
| 2. How it works  | 2–3 sentences | The mechanism or design           | "Monolith and new API both hit the same MySQL. Nginx shifted traffic endpoint by endpoint, so rollback was instant." |
| 3. Proof         | 1 sentence    | A metric or real scenario         | "We moved 100+ tenants with zero downtime."                                                                          |
| 4. Trade-off     | 1 sentence    | The downside or what you'd change | "The cost was tight DB coupling, so every migration had to stay backward-compatible."                                |

## When you don't know or don't remember

| Situation              | Say this                                                                                          | Why it works                                |
|------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------|
| Forgot an exact number | "I don't remember the exact number, but it was roughly [range], and here's how we measured it..." | Moves focus to your method                  |
| Someone else owned it  | "I wasn't directly responsible — [role] owned it — but my understanding is..."                    | Honest about scope, still shows system view |
| You never built it     | "We didn't do that, and it's something I'd add today. Here's how I'd approach it..."              | Shows growth, not defensiveness             |
| Need time to think     | "Good question. Let me think for a second." Then 3 seconds of silence.                            | Replaces "um" with composure                |

## The honesty rule

Never say a number, root cause, tool name or team size you cannot back up. Where this playbook offers options (A / B / C), pick the one that actually happened. Every **[FILL]** marker must be replaced with a verified fact before a real interview; if you can't verify it, drop the claim or soften it to a range. A wrong confident answer costs more than "I'm not sure."

## 2. Defending your numbers

Every number you say will be challenged. For each one, know three things: how it was measured, over what window, and the honest answer to the obvious pushback.

| Claim (say it this way)                                                   | How it was measured                                                  | Likely pushback                  | Your defense                                                                                    |
|---------------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------------------------|
| "Average CPU on core servers dropped about 20% after the migration." (C1) | Grafana / CloudWatch average CPU, 2–4 weeks before vs after cutover; | "Didn't total infra cost go up?" | "Yes, slightly — new services got their own compute. The goal was isolation: report jobs can no |

| Claim (say it this way)                            | How it was measured                                                                     | Likely pushback                            | Your defense                                                                                                                 |
|----------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
|                                                    | billing-surge days excluded                                                             |                                            | longer starve order processing."                                                                                             |
| "Our heaviest report got up to 85% faster." (C2)   | APM traces / EXPLAIN ANALYZE, worst-case run time before vs after                       | "Is 85% the average?"                      | "No — it's the best case on our heaviest report, tens of seconds to low single digits. Average gains were smaller."          |
| "Mean time to resolve dropped roughly 65%." (C3)   | Incident tracker (Jira): alert-to-resolution time, multi-month windows before vs after  | "Maybe the bugs were just easier?"         | "The window was long enough to smooth that out. Three concrete changes drove it: symptom alerts, correlation IDs, runbooks." |
| "Engagement rose about 30% after the new UI." (C4) | Analytics: active users and key actions per session, equal windows pre/post launch      | "Can you attribute it all to the rewrite?" | "Not fully. But no marketing push or big feature shipped at the same time, and faster, simpler flows cut drop-offs."         |
| "Manual stock reconciliation fell about 60%." (C5) | Staff hours logged on manual counts and corrections, before vs after real-time POS sync | "Why not 100%?"                            | "Software removes double-selling. It can't remove theft, damage or mis-scans on the shop floor."                             |
| "Deploys got about 40% faster." (C9)               | CI runner elapsed time, merge to live, manual SSH flow vs Docker pipeline               | "How does adding Docker make it faster?"   | "Layer caching skips dependency installs, and automation removed manual server steps and                                     |

| Claim (say it this way) | How it was measured | Likely pushback | Your defense |
|-------------------------|---------------------|-----------------|--------------|
|                         |                     |                 | checks."     |

Golden rule: say "about" or "up to" unless you have the exact figure and the dashboard to back it.

## C1. Monolith to event-driven microservices

### Pitch (STAR)

- **Situation:** Sokrio ran one Laravel 8 + Vue 2 monolith for 100+ enterprise clients. Month-end reports shared CPU with live order processing, so heavy jobs slowed everyone down and every deploy was risky.
- **Task:** Lead the move to an event-driven architecture with zero downtime for transactions.
- **Action:** Split it into a core Laravel REST API, two React frontends (Admin, Tenant) and three async services — Report, Notification, SOP — connected through RabbitMQ, with MongoDB read models and Redis caching. Cut over in phases, running old and new in parallel.
- **Result:** Zero-downtime cutover, ~20% lower core CPU, and report load fully isolated from order entry.

### How the system fits together

1. Admin and Tenant React apps call the core Laravel API (REST, Sanctum auth).
2. The core API owns orders, users, auth and the MySQL source of truth.
3. It publishes events (e.g. `order.approved`) to RabbitMQ and returns immediately.
4. Report (MongoDB read model), Notification (email/SMS/push) and SOP (rules and its own data) consume those events independently.

### Why these service boundaries

| Rule                    | Question it answers                      | Applied here                                        |
|-------------------------|------------------------------------------|-----------------------------------------------------|
| Different scaling needs | Does it spike differently from the core? | Reports burst at month-end → own service            |
| Natural asynchrony      | Can the user get a 200 before it's done? | Notifications run after <code>order.approved</code> |
| Data ownership          | Can it own its data with no              | SOP owns its rules store;                           |

| Rule | Question it answers      | Applied here                                      |
|------|--------------------------|---------------------------------------------------|
|      | cross-service DB access? | orders/products/tenants stay in the core for ACID |

What stayed together: anything sharing one transaction (orders, products, tenant config) remains a **modular monolith** inside the core. Reporting and notifications are separate **bounded contexts** (DDD) from order management.

### Keeping data in sync during the migration — pick the one that's true [FILL]

| Option                  | How it works                                                                                                                     | Best when                                 |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| A. Shared database      | Monolith and new API use the same MySQL; Nginx moves traffic path by path and can route back instantly                           | Fastest, safest cutover                   |
| B. Events + read models | Core API writes MySQL and publishes events; services build MongoDB projections; history loaded by a one-time idempotent backfill | New services need their own shape of data |
| C. Per-tenant rollout   | Move low-risk tenant databases first, watch metrics, then the next batch                                                         | Database-per-tenant setups                |

### The dual-write problem → Transactional Outbox

If the DB commit succeeds but the publish to RabbitMQ fails (or the reverse), systems drift apart. Fix it by writing the event in the same transaction:

1. BEGIN
2. Insert or update the business row (e.g. orders).
3. Insert the event into `outbox_events` with status `PENDING`.
4. COMMIT — both rows or neither.
5. A worker reads pending rows, publishes to RabbitMQ, and marks them `PROCESSED` after the broker confirms.

Bonus: if RabbitMQ is down, the API keeps working and events wait safely in the outbox.

## Messaging reliability

- **At-least-once delivery → idempotent consumers.** A consumer can crash after doing the work but before its ACK, so the message comes again. Each message carries an `event_id` (UUID); the consumer checks a `processed_messages` table (unique constraint) or a Redis key with TTL and skips duplicates.
- **Retries and DLQ.** Durable queues, persistent messages, ACK only after success. On failure, retry with exponential backoff; after 3–5 tries, route to a dead-letter queue and alert when its depth is above zero.
- **Ordering.** RabbitMQ keeps order only for one queue with one consumer and no retries. Design consumers to read current state from the source, not to trust arrival order.
- **RabbitMQ vs Kafka.** RabbitMQ for task routing, per-message retries and dead-lettering with low ops cost. Kafka for a replayable, high-throughput event log, event sourcing and stream analytics.

## Likely drill questions

**"Why not just scale the monolith?"** — "Scaling the whole monolith would scale everything to handle one bursty workload. Reports spike at month-end while order entry needs steady latency, so separating them let each scale on its own. Order processing stopped slowing down during report runs. The cost was more moving parts to operate and debug."

**"What happens if a consumer processes a message twice?"** — "Nothing, because every consumer is idempotent. Each event has a UUID checked against a unique-constrained table before processing. Duplicates are ACKed and skipped. The trade-off is one extra lookup per message."

**"What would you do differently?"** — "I'd add three things from day one: OpenTelemetry tracing with a correlation ID carried through HTTP and RabbitMQ headers, the outbox pattern on every publisher, and consumer-driven contract tests (e.g. Pact) in CI. Each one closes a gap we only felt in production."

## Say it

- Trade-off: "Microservices gave us independent scaling and deploys, but added operational complexity, distributed debugging and eventual consistency."
- Consistency: "Report data lags the core by a few seconds — fine for analytics; money stays strongly consistent in MySQL."
- Numbers: ~20% lower core CPU · 100+ tenants · 3 services · 0 downtime.

- Phrases: "We ran both systems in parallel to keep instant rollback." · "We used the strangler pattern to replace legacy paths one at a time."

**[FILL]** Main pain point of the monolith · one-line description of SOP · which sync option (A/B/C) · CPU baseline window · dedup key and retry count · team size, duration, your role.

## C2. Database optimization, EXPLAIN and aggregations

### Pitch (STAR)

- **Situation:** Reporting queries ran over tables with millions of rows; the biggest took tens of seconds and timed out.
- **Task:** Get heavy reports fast without hurting live reads and writes.
- **Action:** Profiled slow queries with EXPLAIN / EXPLAIN ANALYZE, added composite indexes in the right column order, removed N+1 queries with eager loading, moved heavy aggregations to MongoDB pipelines, pushed long reports to background queues and cached results in Redis.
- **Result:** Up to 85% faster on the heaviest report; report service held its 99.9% target.

### Reading EXPLAIN

| Column | Bad sign                                                       | Good sign                                     |
|--------|----------------------------------------------------------------|-----------------------------------------------|
| type   | ALL (full table scan), index (full index scan)                 | const, eq_ref, ref, range                     |
| key    | NULL — no index used                                           | The index you expected                        |
| rows   | Close to the table size                                        | Small                                         |
| Extra  | Using temporary; Using filesort — extra temp table / sort pass | Using index — covering index, no table lookup |

EXPLAIN shows the optimizer's plan and estimates without running the query. EXPLAIN ANALYZE (MySQL 8.0.18+) actually runs it and reports real time and real row counts per step — so don't run it on a heavy write query in production.

### Composite indexes

- **Order:** equality columns first, then range or sort columns. For WHERE distributor\_id = 45 AND created\_at >= '2026-01-01', use (distributor\_id,

created\_at) so MySQL jumps to one distributor's slice and scans a sorted date range inside it.

- **Leftmost prefix:** that index helps distributor\_id alone, or both columns — but not created\_at alone. (MySQL 8.0.13+ can sometimes use a *skip scan*, but don't design around it.)
- **Covering index:** add the selected columns, e.g. (distributor\_id, created\_at, status), so the query is answered from the index without reading table rows.
- **Cost:** every index slows INSERT/UPDATE/DELETE and uses space. Keep only indexes that serve frequent, expensive queries.

## Pagination

OFFSET 100000 LIMIT 50 reads and throws away 100,000 rows. Keyset pagination — WHERE id > :last\_seen\_id ORDER BY id LIMIT 50 — costs the same on page 1 and page 2,000.

## Caching and stampedes

- **Key design:** cache:tenant\_12:report\_sales:<md5 of filters> — always tenant-scoped.
- **Invalidation A — TTL only:** e.g. 15 minutes. Simple and reliable; data can be up to 15 minutes stale.
- **Invalidation B — events + TTL:** clear tagged keys when order.updated fires; keep a TTL as a safety net if an event is lost.
- **Stampede:** when a hot key expires, hundreds of requests hit the DB at once. Fix with a Redis lock (SET key val NX EX 10) so one request rebuilds while others wait or get the stale value, and add TTL jitter (e.g. 15 min ± up to 60 s) so keys don't expire together.

## MySQL vs MongoDB for reports

MySQL stays the transactional source of truth. Heavy read-only analytics are projected into MongoDB documents shaped for the report, then queried with a pipeline: \$match (filter on indexed fields, like WHERE) → \$group (totals, like GROUP BY) → \$project (shape output, like SELECT). A read replica takes load off the primary, but the joins are still expensive; pre-shaped documents remove the join work itself.

## Likely drill questions

"Walk me through how you found the slow query." — "I started from the slow query log and ran EXPLAIN on the worst offender. It showed type: ALL and Using filesort,

meaning a full scan plus a sort. A composite index on the filter and sort columns turned it into a range scan, and EXPLAIN ANALYZE confirmed the drop. The trade-off was slightly slower writes on that table."

**"Why not just use a read replica?"** — "A replica moves load off the primary but the query is just as expensive. For our heaviest reports, pre-aggregated MongoDB documents removed the joins entirely. That's what took the worst report from tens of seconds to a few seconds. The cost is keeping a second store in sync."

### Say it

- Trade-offs: "Caching made reports sub-second but added staleness; we agreed the TTL with product." · "Indexes speed reads but add write overhead."
- Numbers: up to 85% faster heaviest report · 99.9% report SLO · millions of rows.
- Phrases: "I measured with EXPLAIN before changing anything." · "Equality columns first, then range."

**[FILL]** Name of the slow report and its before/after seconds · the exact EXPLAIN finding and index added · TTL or event-driven invalidation · how MySQL data reaches MongoDB.

## C3. Incident response, SLOs and alerting

### Pitch (STAR)

- **Situation:** Primary 24/7 on-call for six production apps. During peak traffic, an unindexed query plus a DB load spike slowed the API, backed up queues and broke reports for key clients.
- **Task:** Restore service fast, keep stakeholders informed, find the root cause and prevent a repeat.
- **Action:** An SLO burn alert fired; APM traces and logs pointed to one slow query. Mitigated first (more capacity + query patch), posted updates in a Slack incident channel, then wrote a blameless postmortem and added a query regression check.
- **Result:** Service restored quickly [FILL: exact minutes]. Moving from host alerts to symptom-based SLO alerts helped cut MTTR ~65%.

### SLI → SLO → SLA

| Term            | What it is       | Example                                      |
|-----------------|------------------|----------------------------------------------|
| SLI (indicator) | What you measure | % of requests returning success; p95 latency |
| SLO (objective) | Internal target  | 99.9% success over a rolling 30 days         |

| Term            | What it is                       | Example                                                           |
|-----------------|----------------------------------|-------------------------------------------------------------------|
| SLA (agreement) | External contract with penalties | 99.0% — looser than the SLO so you fix issues before breaching it |

Error budget = 100% – SLO. Over a 30-day month:

| SLO    | Budget | Downtime allowed     |
|--------|--------|----------------------|
| 99%    | 1%     | ~7.2 hours (432 min) |
| 99.9%  | 0.1%   | ~43.2 minutes        |
| 99.99% | 0.01%  | ~4.3 minutes         |

Formula:  $(1 - \text{SLO}) \times 30 \times 24 \times 60$  minutes. When the budget burns fast, pause non-essential releases and put engineers on reliability.

### Alerting that people trust

- **Symptom, not cause.** Page on user impact (5xx rate, p95 > 2 s), not "CPU 85% at 2 AM" when users are fine.
- **Severity tiers:** P1 — users affected, SLO burning → page now. P2 — degraded but covered by redundancy → Slack. P3 — trends and capacity → weekly review.
- **Burn-rate alerts.** Alert on how fast the budget is being spent, not one bad minute. Burning 2% of the monthly budget in 1 hour is a burn rate of ~14x; at that pace the whole budget is gone in about two days → page.

### Incident playbook

1. **Mitigate first, investigate second.** Recent deploy? Roll back. Saturated? Add capacity or restart.
2. **Use runbooks** for known failures (queue backlog, DB connection-pool exhaustion) so anyone on call can act under stress.
3. **Communicate** on a fixed cadence in one channel.
4. **Blameless postmortem:** timeline, customer impact and budget used, root cause (5 Whys), action items with owners and dates.

### Likely drill questions

"Why not target 99.99%?" — "Each extra nine costs far more in redundancy and engineering. 99.9% gave us about 43 minutes a month, which matched what clients actually needed. We hit it consistently. The trade-off is accepting short, rare outages."

"How did you find the slow query so fast?" — "Correlation IDs. The alerting request's ID led straight to its APM trace, which showed one query taking most of the time. That took minutes instead of an hour of log searching. What I'd add now is tracing across the queue as well."

### Say it

- "We alert on user symptoms, not raw compute." · "The error budget tells us when to stop shipping features and fix reliability." · "Every incident ends with a blameless postmortem."
- Numbers: ~65% lower MTTR · 6 apps on call · 99.9% ≈ 43 min/month.

[FILL] The real incident (symptom, root cause, fix, time to restore) · where MTTR baseline came from · names of the six apps.

## C4. Frontend modernization and technical leadership

### Pitch (STAR)

- **Situation:** The frontend was on Vue 2 (end of life), with tangled state, no types and slowing delivery.
- **Task:** Move to React, Next.js, Redux Toolkit and TypeScript in 2 months without freezing feature work.
- **Action:** As frontend tech lead, I limited scope to core user journeys, spent the first two weeks on foundations (routing, API layer, store layout, ESLint/Prettier, shared components), rebuilt key flows from Figma with product/UX, and released in phases.
- **Result:** Shipped on time, set team-wide frontend standards, and engagement rose ~30%.

### How to de-risk a rewrite on a deadline

1. **Cut scope hard.** Use analytics to find the high-value journeys (order entry, dashboards, main reports). Rare admin screens wait.
2. **Foundation first.** Weeks 1–2: API layer, state shape, routing, build tooling, shared components — so people can build pages in parallel without conflicts.
3. **Release journey by journey.** Route at the gateway so new React flows run beside legacy screens. No big-bang launch.

## Which state tool for what

| State                   | Tool                    | Examples                                     |
|-------------------------|-------------------------|----------------------------------------------|
| Local UI                | useState / useReducer   | Modal open, form fields                      |
| Global client state     | Redux Toolkit           | Tenant config, permissions (RBAC)            |
| Server data             | React Query / RTK Query | API cache, refetch, loading and error states |
| Rarely-changing globals | React Context           | Theme, locale                                |

Why not Context for everything: when a Context value changes, every component using that Context re-renders, even if it only reads an unrelated field. Fine for theme; bad for fast-changing data.

## Leading without arguing

- Automate style (ESLint, Prettier, TypeScript strict) so reviews focus on logic, security and architecture.
- Use a PR template with a checklist: tests, types, accessibility.
- In disagreements: listen first, separate preference from real engineering impact, adopt better ideas into the standard, then bake the decision into tooling.

## Likely drill questions

**"Why React instead of upgrading to Vue 3?"** — [FILL: the real reason — e.g. hiring pool, ecosystem, team skills]. Keep it to one sentence of decision, then the mechanism.

**"How did you handle a teammate who disagreed with a standard?"** — "I asked them to walk me through their proposal first. Their folder convention was genuinely cleaner, so we adopted it and encoded it in lint rules. After that, reviews stopped debating structure. The lesson: standards stick when people help write them."

## Say it

- "Rewrites are risky; we contained it with tight scope and incremental releases." · "Strict tooling costs a week of friction and pays back every week after." · "TypeScript costs upfront typing but catches schema mismatches before production."
- Numbers: 2 months · ~30% engagement lift.

**[FILL]** Business reason for Vue → React · analytics tool and exact metric behind 30% · team size and how work was split.

## C5. Multi-tenancy, payments and POS race conditions

### C5-A. Database-per-tenant

**How it works:** a central *landlord* DB holds tenant metadata, domains and billing. Middleware reads the tenant from the domain, subdomain or a header [FILL: which], and switches the DB connection to that tenant's schema for the whole request.

|             | Database-per-tenant                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Biggest win | A forgotten <code>WHERE tenant_id = ?</code> can't leak data — the connection only sees one tenant                                     |
| Other wins  | Per-tenant restore; move a heavy tenant to its own hardware                                                                            |
| Costs       | Migrations run across 100+ DBs and must be backward-compatible; cross-tenant analytics need a warehouse; connection pooling needs care |

#### Tenant context outside HTTP:

- **Queue jobs:** put `tenant_id` in the job payload. The worker switches to that tenant, runs the job, and in a `finally` block switches back to landlord — otherwise the next job may run against the wrong tenant.
- **Redis:** every key is prefixed, e.g. `tenant:104:orders:active`.
- **Access control:** role permissions plus territory scope; service-layer query scopes filter by tenant and the user's territories.

### C5-B. Payment integrations

1. **Verify webhooks.** Recompute HMAC-SHA256 of the *raw* request body with the shared secret and compare with the signature header using a constant-time function (`hash_equals` in PHP, `crypto.timingSafeEqual` in Node) to prevent timing attacks.
2. **Idempotency both ways.** Outgoing: send an idempotency key per payment attempt, so a retry returns the original result instead of charging twice. Incoming: store the gateway transaction ID under a unique index; a duplicate webhook hits the constraint, returns 200 and does nothing.
3. **State machine.** `PENDING` → `PAID` or `PENDING` → `FAILED`. Terminal states never go back, so late or out-of-order webhooks are harmless.

4. **Reconciliation job.** Webhooks can be lost. A scheduled job asks the gateway about anything still PENDING after ~30 minutes and syncs it.

### C5-C. POS inventory race conditions (GirlyShopper)

Online checkout and in-store POS can sell the last unit at the same time. Two fixes:

#### Atomic conditional update (preferred under high concurrency)

```
UPDATE products
SET stock = stock - :qty
WHERE id = :id AND stock >= :qty;
```

The database serializes the two updates. The first changes 1 row; the second finds stock >= qty false and changes 0 rows. Check affected rows and cancel the sale if it's 0.

#### Pessimistic lock (when you need logic between read and write)

```
BEGIN;
SELECT stock FROM products WHERE id = :id FOR UPDATE;
-- check stock in application code
UPDATE products SET stock = stock - :qty WHERE id = :id;
COMMIT;
```

FOR UPDATE puts an exclusive lock on the row. Other transactions trying to *update* or *lock* that row (another FOR UPDATE) wait until commit. Plain SELECTs are not blocked in InnoDB — they read a consistent snapshot.

### Likely drill questions

"What if the webhook arrives before your API call returns?" — "It doesn't matter, because the payment row is created as PENDING before calling the gateway. The webhook finds that row by transaction ID and moves it to PAID; the late API response sees a terminal state and does nothing. Order stops mattering. The reconciliation job covers the case where neither arrives."

"Atomic update or row lock?" — "Atomic update when the check fits in the WHERE clause, because it's one statement and holds the lock for the shortest time. Row lock when there's logic between reading and writing. The cost of row locks is waiting under contention."

### Say it

- "Database-per-tenant costs migration effort but removes cross-tenant leaks at the connection level." · "Webhooks are the fast path; reconciliation is the source of truth." · "Connection switching is reset in the worker's finally block."
- Numbers: 100+ tenant schemas · ~60% less manual reconciliation · 0 double charges [FILL: over what period].

[FILL] Tenant resolution method · payment gateway name · which POS approach you actually used.

## C6. Enterprise API design and mobile connectivity

### Pitch (STAR)

- **Situation:** A major client's ERP integration started failing; their leadership said our API was "intermittently broken."
- **Task:** Find the real cause fast, restore sync and protect the relationship.
- **Action:** Pulled their requests from logs by correlation ID and replayed them in Postman. Root cause: their ERP update started sending dates in a non-standard format, which our validation rejected. Updated the parser to accept both formats, re-processed held orders, improved error messages.
- **Result:** Sync restored, errors stopped, and clearer validation specs prevented repeats.

### Explaining it to non-technical executives: Impact → Cause → Action → Time

| Step   | Example                                                                                       |
|--------|-----------------------------------------------------------------------------------------------|
| Impact | "About 200 orders sent this morning weren't processed."                                       |
| Cause  | "A new date format in your system's updates was rejected by our validation."                  |
| Action | "We now accept both formats and have re-processed all 200 orders."                            |
| Time   | "Everything was back by 10:30 AM, and we added monitoring to catch format changes instantly." |

### REST vs SOAP

|           | REST               | SOAP                        |
|-----------|--------------------|-----------------------------|
| Payload   | JSON (usually)     | XML envelope                |
| Contract  | OpenAPI (optional) | WSDL (standard)             |
| Transport | HTTP/HTTPS         | Mostly HTTP; also SMTP, JMS |

|             | REST                | SOAP                   |
|-------------|---------------------|------------------------|
| Typical use | Web and mobile APIs | Legacy enterprise ERPs |

For legacy clients, expose a thin SOAP adapter that converts XML into the same internal models the REST API uses. For end-of-day batch dumps, accept the request, put it on a queue and process in the background so spikes don't hit the database directly.

### Mobile APIs for weak connectivity

1. **Small payloads** with only the fields the screen needs; paginate every list.
2. **Client-generated UUID** on every write (e.g. check-in). If the response is lost and the app retries, the server sees the same UUID and returns the original result — no duplicates.
3. **Offline queue + batch sync.** The app stores actions in SQLite and sends them together (POST /api/v1/sync/batch) when back online.
4. **Versioning.** You can't force every phone to update. Use /api/v1/, /api/v2/; adding fields is non-breaking, removing or changing them needs a new version.

### Say it

- "Supporting SOAP costs maintenance but lets us sell to clients on legacy ERPs." · "Queue-buffered ingestion adds latency but protects the system from batch spikes."
- Phrases: "I found the defect by pulling the exact correlation ID and replaying the payload." · "Client UUIDs make retries safe on bad networks."

**[FILL]** The real client defect, the order count and the restore time (the numbers above are an example).

## C7. Field-force modules and GPS-spoofing detection

### Pitch (STAR)

- **Situation:** Clients' field teams (sales reps, van distributors, technicians) visit shops. Managers couldn't verify visits, and mock-location apps let staff fake attendance.
- **Task:** Build field modules (Task Manager, Van Sales, Attendance) with server-side GPS-spoofing detection.
- **Action:** Combined device mock-location flags with server checks: geofence radius, travel speed between check-ins, and coordinate-precision anomalies. Suspicious records are flagged for manager review.

- **Result:** Managers trusted attendance data again; the field-force expansion contributed to 27% company revenue growth [FILL: confirm the figure and your share].

### Detection checks (run on the server, in this order)

1. **Device flag.** Android reports mock locations (`Location.isMock()` on API 31+, `isFromMockProvider()` before). The app sends it with the check-in — useful, but a rooted device can hide it, so never rely on it alone.
2. **Geofence.** Distance from the shop's saved coordinates must be within ~50 m, with extra tolerance for indoor and dense-city signal drift.
3. **Impossible travel.** Distance between consecutive check-ins (Haversine formula) ÷ time between them. Faster than ~150 km/h (e.g. 50 km in 2 minutes) → flag.
4. **Too-perfect coordinates.** Real GPS jitters. The exact same coordinates to 7 decimal places across separate check-ins suggests injected locations.

Haversine distance, with  $r$  = Earth's radius (~6,371 km),  $\varphi$  = latitude,  $\lambda$  = longitude:

$$d = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

### Flag, don't auto-reject

Weak signal indoors or between tall buildings makes honest check-ins look wrong. So suspicious check-ins are saved with `flagged_for_review = true` and a reason ("velocity anomaly"). The worker isn't blocked; the manager decides.

### Bridge to Field Nation

Field Nation's technician work orders have the same core problems: proving the technician was on site, working offline for check-in, check-out and signatures, syncing live status to dashboards, and preventing fraudulent completions. Say it directly: "I've built exactly this class of system — location verification, offline sync and fraud flags for field workers."

### Say it

- "Strict rules catch fraud but punish honest staff with bad GPS; flagging balances both." · "Continuous tracking drains battery and data, so we capture location on specific actions instead."
- "Never trust the client alone — validate on the server."
- Numbers: 50 m geofence · 150 km/h speed limit · 27% revenue growth [FILL].

[FILL] Which signals you actually implemented · which modules drove the revenue growth.

## C8. Node.js runtime and async architecture

### Pitch (STAR)

- **Situation:** The platform needed high-concurrency event consumers and real-time routing; long-running PHP worker loops were a poor fit.
- **Task:** Build non-blocking background services beside the Laravel backend.
- **Action:** Built Node.js + Express + TypeScript services to consume RabbitMQ queues and handle real-time events, with centralized error handling and care not to block the event loop.
- **Result:** High concurrency on a small footprint, with real-time work moved off the main web servers.

### The event loop in four lines

1. JavaScript runs on one main thread; synchronous code runs on the call stack.
2. I/O (network, DB) is handed to the OS or libuv's thread pool; Node keeps running other code.
3. When the stack empties, **microtasks** run first: `process.nextTick` callbacks, then resolved Promise callbacks (`.then`, code after `await`).
4. Then the loop moves through its phases for **macrotasks**: timers (`setTimeout`, `setInterval`), I/O callbacks, `setImmediate`.

**Cardinal rule:** never block the main thread. Big synchronous JSON parsing, image processing or tight loops freeze every request. Move them to Worker Threads or a background queue.

### PHP/Laravel vs Node.js

| Workload                                           | Runtime              | Why                                                                         |
|----------------------------------------------------|----------------------|-----------------------------------------------------------------------------|
| Transactional CRUD, complex ORM, admin tools       | PHP / Laravel        | Fast to build; each request is isolated, so one crash doesn't affect others |
| Many open connections, WebSockets, queue consumers | Node.js / TypeScript | Event loop handles thousands of I/O-bound connections with little memory    |

### Error handling in Express

```
// Express 4: async errors don't reach error middleware on their own
const asyncWrapper = (fn) => (req, res, next) =>
```

```
    Promise.resolve(fn(req, res, next)).catch(next);

// Error middleware must declare exactly 4 arguments
app.use((err, req, res, next) => {
  logger.error(err.stack);
  res.status(err.statusCode || 500).json({
    success: false,
    error: err.statusCode ? err.message : 'Internal Server Error',
  });
});

// Node 15+ crashes on unhandled rejections by default – log and shut down
cleanly
process.on('unhandledRejection', (reason) => {
  logger.error('Unhandled rejection', reason);
  server.close(() => process.exit(1));
});
```

Express 5 forwards rejected promises from async handlers to `next(err)` automatically, so the wrapper is no longer needed there. Note the error handler only exposes messages for known errors — raw messages from 500s can leak internals.

### Say it

- "Node is excellent for I/O concurrency, but one CPU-heavy task blocks everyone." · "Running two runtimes adds ops overhead, but each workload gets the right tool."
- "Microtasks like resolved promises run before timers." · "Error middleware needs four arguments."

**[FILL]** One concrete Node service you built: what it consumed, throughput, and what broke once.